Course Syllabus & Training Material: Modern Web Development

- **Total Duration:** 4 Days (3 Hours/Day = 12 Hours total)
- **Target Audience:** Developers learning Modern Full-Stack Web Development with React & Express.js

Course Schedule & Topic Breakdown

Day	Modules & Topics Covered	Exercises / Hands-on Lab
Day 1	**JavaScript Basics & Component Architecture** • ES6+ Features (Destructuring, Spread/Rest, Arrow Functions, Template Literals) • Array & Object manipulation (map, filter, reduce) • Asynchronous JS (Promises, async/await, Fetch API) • React Fundamentals: JSX, Component Design, Props, and Reusability	**JS Exercise 1:** Array & Object Transformation **JS Exercise 2:** Async Data Fetcher & Processor **React Exercise 1:** Employee Profile Card
Day 2	**React State Management & Controlled Forms** • React State with useState • Event Handling & Dynamic UI Updates • Controlled vs. Uncontrolled Components • Dynamic Lists & CRUD Operations in Local State	**React Exercise 2:** Like Button **React Exercise 3:** Employee Registration Form **React Exercise 7:** Student Management System
Day 3	**Hooks, API Integration & Routing** • Side Effects with useEffect • Consuming RESTful APIs using Fetch API • Single Page Applications (SPA) with React Router (BrowserRouter, Routes, Route, Link)	**React Exercise 4:** Posts Viewer **React Exercise 5:** Student Portal

NGIT/KMEC

Day 4	**Global State, Express.js APIs & Project Structure** • React Context API (createContext, useContext) • Global State vs. Local State • Express.js APIs: REST Endpoints, Middleware, Routing, CORS • Full-Stack Project Structure & Best Practices	**React Exercise 6:** Language Switcher **Hands-on:** End-to-End API Integration & Project Architecture Setup

Detailed Training Material & Exercises

Day 1: JavaScript Basics & React Component Design

1. Theory & Core Concepts

- **Modern JavaScript Fundamentals:** ES6 variables (let, const), arrow functions, destructuring objects and arrays, spread/rest operators, template literals.
- **Array Methods:** Iterating and transforming data using .map(), .filter(), and .reduce().
- **Async JavaScript:** Understanding Promises and consuming asynchronous APIs with async/await.
- **React Fundamentals:** Virtual DOM, JSX syntax, component creation, and passing data down hierarchies using props.

JavaScript Exercise 1: Array & Object Transformation

- **Problem Statement:** Create a utility script that accepts an array of employee objects (containing id, name, department, rating, and salary). Using modern array methods (filter, map, and reduce):

    1. Filter employees belonging to the "Engineering" department with a rating >= 4.
    2. Map the filtered list to add a 10% bonus to their salary.
    3. Calculate the total budget needed for all bonused salaries using reduce.

- **Verification Points:**
    1. Filter returns only high-rated Engineering employees.
    2. Transformed objects retain all original keys with an updated salary and a new bonus key.
    3. Calculated total sum is accurate.

JavaScript Exercise 2: Async Data Fetcher & Processor

- **Problem Statement:** Write an asynchronous function using async/await and the Fetch API to retrieve user data from https://jsonplaceholder.typicode.com/users.
    1. Handle loading and error scenarios with try/catch/finally.
    2. Parse the response and extract only name, email, and company.name.
    3. Sort the extracted records alphabetically by name.

- **Verification Points:**

    1. Function gracefully catches network failures.
    2. Extracted array matches the required object structure.
    3. Array is sorted alphabetically by name.

NIELIT/KMEC

React Exercise 1: Employee Profile Card

- **Problem Statement:** Create a React application that displays employee information using reusable components. Develop an Employee component that receives Employee Name, Employee ID, Department, Designation, and Salary as props. Render the Employee component multiple times from App.jsx with different employee details. Each employee should be displayed inside a separate card with proper headings and labels. The application should demonstrate component reusability using props.
- **Playwright Test Cases:**
    1. Verify page heading.
    2. Verify four employee cards.
    3. Verify employee names.
    4. Verify Employee ID.
    5. Verify Department.
    6. Verify Designation.
    7. Verify Salary.

Day 2: React State Management & Controlled Forms

1. Theory & Core Concepts

- **React State (useState):** Declaring state, immutability, updater functions, and re-rendering lifecycles.
- **Event Handling:** Synthetic events (onClick, onChange, onSubmit).
- **Controlled Components:** Managing form inputs via React state for complete input lifecycle management.
- **List Management & Keys:** Rendering dynamic lists and performing client-side CRUD operations (adding and removing items by unique IDs).

React Exercise 2: Like Button

- **Problem Statement:** Create a Like Button application using the useState Hook. Display the title 'Like Button Application' along with the current number of likes. Initially, the likes count should be 0. Clicking the Like button should increase the count by one, while clicking the Reset button should reset the count back to zero. Use event handling and state management to update the UI dynamically.
- **Playwright Test Cases:**
    1. Verify heading.
    2. Initial likes is 0.
    3. Like button.
    4. Reset button.
    5. Like once.
    6. Like three times.
    7. Reset returns 0.

React Exercise 3: Employee Registration Form

- **Problem Statement:** Create an Employee Registration Form using controlled components. Accept Employee Name, Employee ID, Department, Designation, and Salary from the user. Manage all input fields using the useState Hook. When the Add Employee button is clicked, display the entered employee information below the form in a well-formatted section. The form should update the displayed details dynamically.
- **Playwright Test Cases:**
    1. Verify heading.
    2. Five input fields.
    3. Add button.

NSTKMEC

4. Input accepts values.
5. Employee details displayed.
6. Name displayed.
7. ID displayed.

React Exercise 7: Student Management System

- **Problem Statement:** Create a Student Management System using the useState Hook. Accept Student Name, Roll Number, and Branch from the user. Initially display 'No Students Found'. Clicking Add Student should insert the student into a table. Each row should contain a Delete button to remove that student. The UI should update automatically whenever records are added or deleted.
- **Playwright Test Cases:**
    1. Verify heading.
    2. No Students Found initially.
    3. Three inputs.
    4. Add Student button.
    5. Add one student.
    6. Add multiple students.
    7. Delete student.

Day 3: Lifecycle Hooks, API Integration & Routing

1. Theory & Core Concepts

- **Effects in React (useEffect):** Side-effect management, dependency arrays, API calls on component mount, and cleanup functions.
- **Asynchronous Integration:** Handling loading states, error states, and rendering dynamic tabular data.
- **Client-Side Routing (react-router-dom):** Single Page Application concept, configuring BrowserRouter, Routes, Route, and seamless navigation via <Link> without page reloads.

React Exercise 4: Posts Viewer

- **Problem Statement:** Create a React application that retrieves post data from https://jsonplaceholder.typicode.com/posts using the Fetch API. Fetch the data inside the useEffect Hook and store it using useState. Display all posts in a table with Post ID, Title, and Description columns. Ensure the posts are loaded automatically when the application starts.
- **Playwright Test Cases:**
    1. Verify heading.
    2. Table visible.
    3. Three headers.
    4. Posts loaded.
    5. First ID.
    6. First title.
    7. 100 rows displayed.

React Exercise 5: Student Portal

- **Problem Statement:** Develop a Student Portal using React Router. Create four pages: Dashboard, Profile, Attendance, and Marks. Provide navigation links that allow users to move between these pages without refreshing the browser. Configure routing using BrowserRouter, Routes, Route, and Link from react-router-dom.

NOTKMEC

- **Playwright Test Cases:**
    1. Verify heading.
    2. Dashboard link.
    3. Profile navigation.
    4. Attendance navigation.
    5. Marks navigation.
    6. Profile details.
    7. Marks displayed.

Day 4: Context API, Express.js APIs & Project Structure

1. Theory & Core Concepts

- **React Context API:** Solving prop drilling, creating contexts with createContext, supplying state via <Context.Provider>, and consuming it with useContext.
- **Express.js APIs:** Building lightweight REST backends, route handling (GET, POST, PUT, DELETE), request parsing middleware (express.json()), and enabling CORS for frontend connectivity.
- **Full-Stack Project Structure:** Structuring scalable projects (separating components, pages, context, hooks, routes, and controllers).

React Exercise 6: Language Switcher

- **Problem Statement:** Create a Language Switcher application using the React Context API. Display a welcome message and provide two buttons: English and Telugu. Clicking each button should update the displayed message in the corresponding language. Share the selected language between components using createContext and useContext.
- **Playwright Test Cases:**
    1. Verify heading.
    2. English button.
    3. Telugu button.
    4. Default English message.
    5. Switch to Telugu.
    6. Switch back to English.
    7. Only one message displayed.

Hands-on Lab: Express.js Backend & Full-Stack Project Structure

- **Problem Statement:** Create an Express.js backend server with endpoints for GET /api/students and POST /api/students. Connect this API to the React application developed in **Exercise 7**, organizing both projects into a clear directory architecture:
- Plaintext

```
root/
├── client/          # React Application (Vite / CRA)
│   ├── src/
│   │   ├── components/ # Reusable UI Components
│   │   ├── context/    # Context Providers
│   │   ├── pages/      # Routed Page Views
│   │   └── App.jsx
│   └── package.json
└── server/          # Express Application
    ├── routes/      # Express API Routes
    ├── controllers/   # Route Handlers
    ├── server.js      # App entry point & middleware
    └── package.json
```

NOT/KMEC

Course Introduction: Journey Into Modern Web Development

Think about your daily digital routine. When you open **Amazon** or **Flipkart** to buy shoes, check your feed on **Instagram**, or manage your grades on a university **Student Portal**, you are interacting with dynamic web applications.

Every single click—like filtering products by price, hitting a "Like" heart button, switching languages, or navigating between a profile page and a dashboard—happens because of a carefully structured blend of front-end web components, background data processing, and server communication.

This 4-day intensive course takes you behind the scenes of these everyday web applications. Instead of starting with dry definitions, let's explore how the web works by looking at daily digital experiences, and then discover the core concepts that power them.

Step 1: From Everyday Web Pages to JavaScript Basics & Components

- **Real-World Experience:** Imagine building a digital shopping cart or an employee directory card. To display lists of staff members or products neatly, you need reusable blocks of design, just like individual product cards repeating on an e-commerce grid.
- **The Underlying Concepts:** * **Modern JavaScript:** You will learn how to write clean ES6+ code (arrow functions, destructuring objects) and manipulate arrays (map, filter, reduce) to filter out items or calculate totals.
    - **Asynchronous Operations:** Just like waiting for an app to load fresh data from a cloud server, you will master async/await and the Fetch API to pull live data.
    - **Component Architecture:** You will discover how React lets you build reusable pieces of UI (like an Employee profile card) and pass data down using **Props**.

Step 2: From Tapping "Like" Buttons to React State Management

- **Real-World Experience:** Think of tapping a "Like" button on a social media post where the counter instantly ticks up from 0 to 1, or filling out a registration form where your inputs dynamically appear on screen.

- **The Underlying Concepts:**

    - **React State (useState):** This concept tracks data that changes over time within a component.

    - **Controlled Components & Forms:** You will learn how user inputs bind directly to application memory, allowing you to instantly build dynamic lists, tables, and CRUD (Create, Read, Update, Delete) dashboards like a *Student Management System*.

Step 3: From Seamless Page Navigation to Routing & Lifecycle Hooks

- **Real-World Experience:** When you use a banking app or a university portal, clicking tabs like *Dashboard*, *Profile*, *Attendance*, or *Marks* switches views instantly *without* the web browser flashing or doing a full page reload.

- **The Underlying Concepts:**

NGTWIEC

- **Lifecycle Side Effects (useEffect):** How applications automatically fetch data or trigger actions right when a page loads.

- **Single Page Applications (React Router):** Using BrowserRouter, Routes, and Link to create seamless client-side navigation that mimics native desktop applications.

Step 4: From Global Language Toggles to Express.js Backends

- **Real-World Experience:** Have you ever clicked a button to switch an entire website's language from English to Telugu, or logged into an app where your user settings persist across every page? Behind the scenes, servers are handling requests and sending back data.

- **The Underlying Concepts:**

    - **React Context API:** Shares global states (like active language preferences or user themes) across deeply nested component trees without manual prop-drilling.

    - **Express.js APIs:** Building lightweight server-side code and REST endpoints (GET, POST) so your React frontend can communicate with a database and securely store user or student records.

NGIT/KMEC

📜 What is a Scripting Language & What is JavaScript?

1. What is a Scripting Language?

A **scripting language** is a type of programming language that translates or executes instructions **on the fly** (at runtime) rather than requiring a separate compilation step beforehand.

- **How it works:** Traditional compiled languages (like C++ or Java) require you to run a compiler that converts your human-readable source code into a machine-executable binary file before you can run it. Scripting languages usually use an **interpreter** that reads, analyzes, and executes the script line by line directly inside a host environment (like a web browser or a server).

- **Common Uses:** Automating repetitive tasks, powering web browser interactivity, building server-side applications, and writing system administration utilities (e.g., Python, JavaScript, Bash, and Ruby).

2. What is JavaScript?

JavaScript (JS) is a lightweight, interpreted, high-level programming language primarily known as the scripting language for the Web.

- **What it does:** Alongside HTML (structure) and CSS (styling), JavaScript is the third core layer of the World Wide Web. It gives web pages **interactivity and behavior**—enabling dynamic form validations, animations, pop-up modals, live data updates (via APIs), and full-fledged web applications (like React, Vue, or Angular dashboards).

3. How is JavaScript Related to Java?

Despite their strikingly similar names, **Java and JavaScript are entirely different programming languages** created by different companies for entirely different purposes.

The naming similarity is largely a historical marketing decision made back in the 1990s.

Key Differences at a Glance

Feature	Java	JavaScript
Creator	James Gosling at **Sun Microsystems** (1995)	Brendan Eich at **Netscape** (1995)
Language Type	**Compiled** object-oriented programming language	**Interpreted** scripting language
Execution Environment	Runs on the **Java Virtual Machine (JVM)**	Runs inside web **browsers** and **Node.js** runtimes
Typing	**Statically typed** (you must declare data types like int, String)	**Dynamically typed** (variables can hold any data type using let, const)
Primary Use Case	Enterprise backend systems, Android apps, large-scale desktop software	Web frontend interactivity, full-stack web apps, server-side scripting

Analogy: As the famous saying goes: *"Java is to JavaScript as Car is to Carpet."* Aside from sharing some structural syntax roots inspired by C, they are completely separate technologies with entirely different ecosystems.

🖨️ Basic Printing and Display Statements in JavaScript

NSTKMEC

Unlike some other languages that use a single command (like print() in Python or System.out.println() in Java), JavaScript provides several different ways to output or "print" information depending on **where** you want the output to appear.

1. console.log() (For Developers & Debugging)

This is the most common way to print messages in JavaScript. It outputs text, variables, or data objects directly to the **developer console** (the browser console or your terminal if using Node.js). It does not affect what the average user sees on the actual webpage.

```
JavaScript
// Printing a simple string
console.log("Hello, World!");

// Printing variables and numbers
let userName = "Priya";
let userScore = 95;
console.log("User:", userName, "Score:", userScore);
```

2. document.write() (Directly to the Web Page)

This method writes HTML or text directly into the HTML document stream. If executed after the document has finished loading, it will wipe out the existing page content. Because of this, it is rarely used in modern professional web development, but it is great for quick learning scripts.

```
JavaScript
// Writes text directly onto the rendered web page body
document.write("<h1>Welcome to my website!</h1>");
document.write("<p>This text was written using JavaScript document.write.</p>");
```

3. Modifying DOM Elements (The Professional Way to Display on Screen)

In modern web applications, instead of writing raw text onto the page, you target a specific HTML element (like a or <p>) using its id or class, and change its text content dynamically.

- **Using textContent:** Safely updates text inside an element.
- **Using innerHTML:** Allows you to inject both text and HTML tags.

Example:

```
HTML
<p id="output-message"></p>

<script>
  // JavaScript to display text inside that specific paragraph element
  let greetingMessage = "Hello! This is rendered dynamically inside a DOM element.";

  document.getElementById("output-message").textContent = greetingMessage;
</script>
```

NGTKMEC

4. window.alert() (Popup Notification Box)

This method opens a modal alert box with a specified message and an OK button. It pauses the execution of the page until the user clicks "OK". It is usually used for important warnings or alerts.

JavaScript
// Triggers a browser pop-up box
window.alert("Warning: Your session is about to expire!");

5. window.prompt() & window.confirm() (Interactive User Input/Display)

- **prompt():** Opens a dialog box asking the user to input text.
- **confirm():** Opens a dialog box with OK and Cancel buttons, returning true or false based on the user's choice.

JavaScript
// Prompt the user for their name and print it
let userResponse = window.prompt("Please enter your name:");
console.log("User entered: " + userResponse);

// Ask for confirmation
let isConfirmed = window.confirm("Are you sure you want to delete this file?");
console.log("User choice: " + isConfirmed);

📦 Deep Dive: var, let, and const in JavaScript

Building on how we display information, let's look closely at how JavaScript stores data. JavaScript provides three keywords to declare variables: **var**, **let**, and **const**.

Understanding how they differ in **scope**, **reassignment**, and **hoisting** is crucial for writing clean code.

1. The Core Differences at a Glance

Feature	var	let	const

NGITKMEC

Introduction	Old standard (ES5 and older)	Modern standard (ES6+)	Modern standard (ES6+)
Scope Type	Function-scoped	Block-scoped ({})	Block-scoped ({})
Reassignable?	Yes	Yes	**No** (Throws error on reassignment)
Redeclarable?	Yes (within same scope)	No	No
Hoisting Behavior	Hoisted & initialized as undefined	Hoisted but *uninitialized* (Temporal Dead Zone)	Hoisted but *uninitialized* (Temporal Dead Zone)

NGTKMEC

2. Scope: Function-Scoped (var) vs. Block-Scoped (let & const)

Scope refers to the visibility or accessibility of variables in your code.

A. var is Function-Scoped

If you declare a variable using var inside a function, it is available anywhere inside that function. However, if you declare it inside a block (like an if statement or a for loop wrapped in curly braces {}), **it leaks out of that block** into the surrounding function or global scope.

```
JavaScript
function testVarScope() {
  if (true) {
    var message = "Hello from inside the block";
  }
  // It leaks outside the if block!
  console.log(message); // Output: "Hello from inside the block"
}
testVarScope();
```

B. let and const are Block-Scoped

Both let and const respect block boundaries. Any code inside curly braces {} creates a "block scope". Variables declared with let or const inside a block **cannot** be accessed outside of it.

JavaScript
```
function testLetScope() {
  if (true) {
    let blockMessage = "Hello strictly inside block";
    console.log(blockMessage); // Works fine here
  }

  // Trying to access it outside the block throws a ReferenceError:
  // console.log(blockMessage);
}
testLetScope();
```

3. Reassignment & Mutability Rules

- **var & let allow reassignment:** You can change their values as many times as you want.

- JavaScript

```
let score = 10;
score = 20; // Perfectly valid
```

-
- **const blocks reassignment:** Once you assign a value to a const variable, you cannot change it. Trying to reassign it triggers a TypeError.

- JavaScript

```
const maxLimit = 100;
// maxLimit = 200; // Error: Assignment to constant variable.
```

-
  Important Note on const with Objects/Arrays: > const prevents *reassignment* of the variable identifier itself, but it does **not** make objects or arrays immutable. You can still modify properties or add elements inside a const collection.

  JavaScript

```
const user = { name: "Priya" };
user.name = "Aarav"; // Valid! We are mutating the object property, not reassigning 'user'.
```

4. Hoisting Behavior

Hoisting is JavaScript's default behavior of moving variable declarations to the top of their scope during the compilation phase before code execution.

- **var Hoisting:** A var variable is hoisted to the top and automatically initialized with a value of undefined. You can technically use it before it is declared (though it will return undefined instead of throwing an error).

- JavaScript

```
console.log(userName); // Output: undefined (no error thrown)
var userName = "Rahul";
```

-
- **let & const Hoisting (Temporal Dead Zone):** They are hoisted to the top of their block, but they are **not initialized**. Accessing them before the line where they are declared

NGIT/KMEC

throws a ReferenceError. The time between entering the scope and the actual declaration is called the **Temporal Dead Zone (TDZ)**.

- JavaScript

```
// console.log(userId); // ReferenceError: Cannot access 'userId' before initialization
let userId = "EMP102";
```

-

Summary Rule of Thumb for Modern Development

1. Use **const** by default for almost everything (variables and functions that do not need to be reassigned).

2. Use **let** only when you know the value of the variable will need to change later (like a counter in a loop or toggle state).

3. Completely avoid **var** in modern JavaScript codebases to prevent scoping bugs and unpredictable behavior.

🏹 Arrow Functions in ES6 JavaScript

Continuing our journey into modern **ES6+ features**, let's look at **Arrow Functions**. Introduced in ES6 (2015), arrow functions provide a much shorter, cleaner syntax for writing function expressions in JavaScript.

1. Traditional Functions vs. Arrow Functions

To understand why arrow functions are so popular, let's compare how you write a simple function that adds two numbers.

The Traditional Way (Function Expression)

JavaScript

```
const addNumbers = function(a, b) {
  return a + b;
};

console.log(addNumbers(5, 3)); // Output: 8
```

The Modern Arrow Function Way

With arrow functions, you drop the function keyword and use a "fat arrow" (=>) between the parameters and the code block.

JavaScript

```
const addNumbersArrow = (a, b) => {
  return a + b;
};

console.log(addNumbersArrow(5, 3)); // Output: 8
```

NGIT/KMEC

2. Shorthand Syntax (Implicit Return)

Arrow functions have a special superpower: **implicit return**. If your function consists of just a single line of code that evaluates and returns a value, you can:

1. **Remove the curly braces {}**
2. **Remove the return keyword**

JavaScript will automatically return the result of that single expression.

JavaScript

```
// Long arrow function with explicit return
const multiplyByTwo = (num) => {
  return num * 2;
};

// Shorthand arrow function with implicit return (Clean & Concise!)
const multiplyByTwoShort = num => num * 2;

console.log(multiplyByTwoShort(4)); // Output: 8
```

> **Parameter Rule:** If your arrow function takes **only one parameter** (like num above), you can even omit the parentheses around the parameter list. If it has zero parameters or two-plus parameters, parentheses are required (() => ... or (a, b) => ...).

3. Real-World Use Case: Array Mapping

Arrow functions shine brightest when used alongside functional array methods like .map(), .filter(), and .reduce().

Imagine you have an array of product prices and want to apply a discount:

JavaScript

```
const originalPrices = [1000, 2500, 4000];

// Using arrow function inside map makes data transformation effortless
const discountedPrices = originalPrices.map(price => price * 0.9);

console.log(discountedPrices); // Output: [900, 2250, 3600]
```

4. One Important Difference: The this Keyword

Aside from shorter syntax, arrow functions behave differently regarding the this keyword:

- **Traditional functions** define their own this context dynamically depending on how they are called.
- **Arrow functions** do **not** have their own this. Instead, they inherit this from their surrounding parent scope (known as *lexical scoping*). This makes arrow functions

NGIT/KMEC

extremely useful inside callbacks (like inside setTimeout or event handlers) where you want this to refer to your main component object instead of the global window.

📦 Destructuring Objects and Arrays in ES6 JavaScript

Destructuring is a powerful ES6 feature that allows you to unpack values from arrays or properties from objects into distinct variables cleanly and concisely. Instead of accessing properties one by one using dot notation or index numbers, destructuring lets you extract them in a single, readable line of code.

1. Object Destructuring

When working with APIs or configuration objects, you often receive large objects but only need a few properties.

The Old Way vs. The ES6 Destructuring Way

JavaScript

```
const userProfile = {
  id: "EMP001",
  name: "Priya Sharma",
  email: "priya@company.com",
  role: "Frontend Developer"
};

// --- THE OLD WAY ---
let nameOld = userProfile.name;
let emailOld = userProfile.email;

// --- THE ES6 DESTRUCTURING WAY ---
// The variable names MUST match the property keys in the object
const { name, email } = userProfile;

console.log(name);  // Output: Priya Sharma
console.log(email); // Output: priya@company.com
```

Key Object Destructuring Features:

NGIT/KMEC

1. **Renaming Variables:** You can assign extracted properties to completely new variable names using a colon (`:`).

2. JavaScript

```
const { name: fullName, role: designation } = userProfile;
console.log(fullName);    // Output: Priya Sharma
console.log(designation); // Output: Frontend Developer
```

3.
4.
5. **Default Values:** If a property might be missing from the object, you can set a fallback default value.

6. JavaScript

```
const { name, department = "Engineering" } = userProfile;
console.log(department); // Output: Engineering (since department wasn't in userProfile)
```

7.

8.

2. Array Destructuring

Array destructuring allows you to unpack values from arrays based on their **position (index order)** rather than their names.

The Old Way vs. The ES6 Destructuring Way

JavaScript

```
const rgbColor = [255, 128, 0];

// --- THE OLD WAY ---
let red = rgbColor[0];
let green = rgbColor[1];
let blue = rgbColor[2];

// --- THE ES6 DESTRUCTURING WAY ---
// Variable names can be anything you choose, ordered left-to-right
const [r, g, b] = rgbColor;

console.log(r); // Output: 255
console.log(g); // Output: 128
console.log(b); // Output: 0
```

Key Array Destructuring Features:

1. **Skipping Items:** You can skip items you don't need by leaving an empty space with a comma.

NGIT/KMEC

2. JavaScript

```
const scores = [95, 82, 88];
const [firstScore, , thirdScore] = scores; // Skipped index 1 (82)
console.log(firstScore, thirdScore); // Output: 95 88
```

3.
4.
5. **Using Rest Operator (`...`):** You can collect the remaining items of an array into a separate sub-array.

6. JavaScript

```
const stack = ["React", "Node.js", "Express", "MongoDB"];
const [primaryFrontend, ...backends] = stack;

console.log(primaryFrontend); // Output: React
console.log(backends);        // Output: ["Node.js", "Express", "MongoDB"]
```

7.

8.

3. Real-World Example: E-Commerce User Dashboard

Here is how destructuring comes together in a real-world component function that processes a nested API response:

JavaScript

```
// A nested response object from an e-commerce backend
const apiResponse = {
  status: 200,
  data: {
    customer: {
      username: "Rahul_Dev",
      billingEmail: "rahul@mail.com",
      shippingAddress: {
        city: "Hyderabad",
        pincode: "500081"
      }
    }
  }
};

// Clean nested object destructuring
const { customer: { username, billingEmail, shippingAddress: { city } } } = apiResponse.data;

console.log(`User: ${username}, Email: ${billingEmail}, City: ${city}`);
// Output: User: Rahul_Dev, Email: rahul@mail.com, City: Hyderabad
```

NGIT/KMEC

🔄 Spread and Rest Operators (...) in ES6 JavaScript

Both the **Spread** operator and the **Rest** operator use the exact same syntax—three dots (...)—but they perform completely opposite actions depending on *where* and *how* you use them.

1. The Spread Operator (...): Unpacking Elements

The spread operator **expands or unpacks** elements out of arrays or object literals. Think of it as taking everything *inside* a container and spreading individual items out where they are needed.

A. Spreading Arrays (Merging / Cloning)

JavaScript

```
const frontendSkills = ["HTML", "CSS", "JavaScript"];
const fullStackSkills = [...frontendSkills, "React.js", "Node.js"];

console.log(fullStackSkills);
// Output: ["HTML", "CSS", "JavaScript", "React.js", "Node.js"]
```

B. Spreading Objects (Copying & Updating Properties)

In modern React and component architecture, you often need to update state immutably. The spread operator allows you to copy existing properties and override specific ones cleanly.

JavaScript

```
const userProfile = { name: "Priya", role: "Developer", active: true };

// Creating a new object by copying userProfile and updating 'active'
const updatedProfile = { ...userProfile, active: false, department: "Engineering" };

console.log(updatedProfile);
// Output: { name: 'Priya', role: 'Developer', active: false, department: 'Engineering' }
```

2. The Rest Operator (...): Packing Elements

The rest operator does the exact **opposite** of spreading: it **collects multiple elements together and packs them into a single array**. It is typically used in function parameter definitions or destructuring assignments to handle an indefinite number of arguments.

A. Collecting Function Arguments

If you are writing a utility function that accepts any number of numerical scores or items, use the rest parameter to catch all remaining arguments.

JavaScript

```
function calculateCartTotal(customerName, ...itemPrices) {
  // itemPrices is an array containing all remaining arguments passed in
  const total = itemPrices.reduce((sum, price) => sum + price, 0);
  return `${customerName}, your total is ₹${total}`;
}

console.log(calculateCartTotal("Aarav", 499, 1299, 250));
// Output: Aarav, your total is ₹2048
```

NGITKMEC

B. Rest in Destructuring

You can use the rest operator during array or object destructuring to extract specific items and pack everything else into a separate variable.

JavaScript

```
const numbers = [10, 20, 30, 40, 50];

// Extract the first item, and pack the rest into a new array
const [first, ...remainingNumbers] = numbers;

console.log(first);            // Output: 10
console.log(remainingNumbers);  // Output: [20, 30, 40, 50]
```

🆚 Quick Summary Comparison

Feature	Spread Operator (...)	Rest Operator (...)
What does it do?	**Unpacks / Expands** items out	**Packs / Collects** items together
Where is it used?	Inside array literals, object literals, or function call arguments	Inside function parameter lists or destructuring assignments
Analogy	Emptying a box of chocolates out onto a table	Gathering scattered chocolates back into a single box

📦 Object Rest in Destructuring

When you destructure an object, you can extract specific properties you want to use immediately, and pack **all remaining properties** into a brand-new object using the rest operator (...).

Real-World Example: E-Commerce Product Card Component

Imagine you are building a product component. You want to extract the id and name to display them directly, but you want to pack all other metadata properties (like price, rating, vendor) into a separate object to pass down or handle separately.

JavaScript

```
// A full product object from your database
const product = {
  id: "PROD-501",
  name: "Wireless Noise-Cancelling Headphones",
  price: 5999,
  rating: 4.5,
  vendor: "AudioTech Inc.",
```

NGIT/KMEC

```
  inStock: true
};

// --- OBJECT REST DESTRUCTURING ---
// We pull out 'id' and 'name', and pack everything else into 'remainingDetails'
const { id, name, ...remainingDetails } = product;

console.log(id);              // Output: PROD-501
console.log(name);            // Output: Wireless Noise-Cancelling Headphones
console.log(remainingDetails);
// Output: { price: 5999, rating: 4.5, vendor: 'AudioTech Inc.', inStock: true }
```

Array Rest vs. Object Rest Summary

Destructuring Type	What is unpacked?	What does the Rest operator (...) collect into?
Array Rest (const [first, ...rest] = array)	Elements by **position / index**	A **new Array** containing all remaining elements
Object Rest (const { key1, ...rest } = object)	Properties by **key name**	A **new Object** containing all remaining key-value pairs

NGIT/KMEC

Template Literals in ES6 JavaScript

Template Literals (introduced in ES6) are a modern, powerful way to work with strings in JavaScript. They use **backticks** (`) instead of traditional single quotes (') or double quotes (") and solve many of the limitations of old string concatenation.

1. Why Do We Need Template Literals?

Before ES6, if you wanted to combine variables and text into a string, you had to use the plus sign (+) operator. As strings grew longer or required multiple variables, the code became messy, hard to read, and prone to syntax errors with missing spaces or quotes.

The Old Way (String Concatenation)

JavaScript

```
const firstName = "Priya";
```

```
const role = "Frontend Developer";
const experience = 3;

// Clunky and easy to miss spacing errors
const oldMessage = "Hello " + firstName + ", you are a " + role + " with " + experience + " years of experience.";
console.log(oldMessage);
```

The Modern ES6 Way (Template Literals)

With template literals, you simply enclose your string in backticks and embed your variables directly using the **dollar sign and curly braces syntax** (`${variableName}`).

```
JavaScript
const firstName = "Priya";
const role = "Frontend Developer";
const experience = 3;

// Clean, readable, and intuitive!
const modernMessage = `Hello ${firstName}, you are a ${role} with ${experience} years of experience.`;
console.log(modernMessage);
// Output: Hello Priya, you are a Frontend Developer with 3 years of experience.
```

2. Key Features of Template Literals

A. Embedding Expressions & Calculations

You aren't restricted to just putting variables inside `${}`. You can evaluate any valid JavaScript expression or mathematical operation inline.

```
JavaScript
const itemPrice = 1200;
const taxRate = 0.18;

const receiptMessage = `Your final bill amount is ₹${itemPrice + (itemPrice * taxRate)}.`;
console.log(receiptMessage);
// Output: Your final bill amount is ₹1416.
```

B. Multi-Line Strings Without Special Characters

In old JavaScript, if you wanted a string to span multiple lines, you had to use the newline escape character (`\n`). Template literals natively support multi-line strings simply by hitting `Enter` on your keyboard.

```
JavaScript
// Multi-line HTML markup generation for a UI component
const cardTemplate = `
```

NGIT/KMEC

```
  <div class="user-card">
    <h2>Aarav Sharma</h2>
    <p>Department: Engineering</p>
  </div>
`;

console.log(cardTemplate);
```

3. Real-World Use Case: Dynamic Component Markup

In modern frontend architecture and React-like thinking, template literals are heavily used to generate dynamic HTML snippets, log formatted messages, or build clean URL endpoint paths.

```
JavaScript
const userId = "EMP-4092";
const category = "electronics";

// Clean API endpoint string construction
const apiEndpoint = `https://api.store.com/v1/users/${userId}/purchases?category=${category}`;

console.log(apiEndpoint);
// Output: https://api.store.com/v1/users/EMP-4092/purchases?category=electronics
```

🔄 Array & Object Manipulation (`map`, `filter`, `reduce`)

In modern web development and component-driven architecture, functional array methods (`map`, `filter`, and `reduce`) are the gold standard for processing data collections. Instead of mutating original arrays with traditional `for` or `while` loops, these methods allow you to transform data immutably and cleanly.

1. The Big Picture: Why Functional Methods?

When rendering dynamic lists in a dashboard or processing an e-commerce shopping cart, you want your data operations to be:

- **Predictable:** They return new arrays or values instead of altering the original data source.
- **Declarative:** You specify *what* transformation to perform rather than writing verbose indexing logic.

NGIT/KMEC

2. `.filter()` — Selecting Elements Based on a Condition

The `filter()` method iterates through an array, runs a test callback function on each element, and returns a **brand-new array** containing only the elements that pass the test (`true`).

Example: Keeping Only In-Stock Products

JavaScript

```
const inventory = [
  { id: 1, name: "Laptop", inStock: true },
  { id: 2, name: "Tablet", inStock: false },
  { id: 3, name: "Smartphone", inStock: true }
];

// Keep only items where inStock is true
const availableItems = inventory.filter(item => item.inStock);

console.log(availableItems);
// Output: [{ id: 1, name: 'Laptop', inStock: true }, { id: 3, name: 'Smartphone', inStock: true }]
```

3. `.map()` — Transforming Every Element

The `map()` method transforms every element in an array by calling a provided function on each item and returning a **brand-new array of the exact same length**.

Example: Applying a Regional Tax to Prices

JavaScript

```
const basePrices = [1000, 2000, 3500];

// Map over each price to include an 18% tax
const taxedPrices = basePrices.map(price => price * 1.18);

console.log(taxedPrices);
// Output: [1180, 2360, 4130]
```

4. `.reduce()` — Accumulating to a Single Value

The `reduce()` method executes a reducer function on each element of the array, resulting in a **single accumulated output value** (which can be a number, a string, an object, or a nested array).

How `reduce()` Syntax Works:

JavaScript

```
array.reduce((accumulator, currentItem) => {
```

NCITKMEC

```
  // return updated accumulator
}, initialValue);
```

Example: Calculating the Final Checkout Total

JavaScript

```
const cartItems = [
  { name: "Mouse", price: 800 },
  { name: "Keyboard", price: 2500 },
  { name: "USB Hub", price: 1200 }
];

// Accumulate the sum of all item prices starting from an initial total of 0
const checkoutTotal = cartItems.reduce((accumulator, item) => accumulator + item.price, 0);

console.log("Final Checkout Total: ₹", checkoutTotal);
// Output: Final Checkout Total: ₹ 4500
```

5. Real-World Combined Pipeline: The Shopping Cart Workflow

In real production applications, you will often chain these methods together into a powerful processing pipeline.

JavaScript

```
// Real-World Example: Processing an e-commerce shopping cart
const shoppingCart = [
  { id: 101, name: "Wireless Earbuds", price: 2000, inStock: true },
  { id: 102, name: "Mechanical Keyboard", price: 4000, inStock: false }, // Out of stock
  { id: 103, name: "Type-C Adapter", price: 800, inStock: true }
];

// Step 1: Filter out-of-stock items -> Step 2: Map prices with tax -> Step 3: Reduce to total
const finalCartTotal = shoppingCart
  .filter(item => item.inStock)                    // Keep only inStock: true
  .map(item => ({ ...item, taxedPrice: item.price * 1.1 })) // Add 10% tax property
  .reduce((sum, item) => sum + item.taxedPrice, 0);     // Sum up final prices

console.log("Processed Shopping Cart Total with Tax: ₹", finalCartTotal);
// Output: Processed Shopping Cart Total with Tax: ₹ 3080
```

NGIT/KMEC

⏱️ Asynchronous JavaScript (Promises, `async`/`await`, Fetch API)

JavaScript is **single-threaded**, meaning it can only execute one line of code at a time on its main call stack. If JavaScript had to wait synchronously for slow operations—like downloading a large file from a server, reading a database, or fetching live crypto market prices—the entire web browser would freeze and become completely unresponsive.

To solve this, JavaScript uses **Asynchronous Programming**. It allows tasks to run in the background without blocking the main thread, handling results whenever they become available.

1. Understanding Asynchronicity: Synchronous vs. Asynchronous

- **Synchronous (Blocking):** Code runs sequentially. Line 2 cannot execute until Line 1 has completely finished executing.
- **Asynchronous (Non-Blocking):** Long-running tasks are sent off to the browser's background environment (or web APIs), allowing the rest of your program to keep running immediately. Once the background task finishes, its result is pushed back into the execution flow.

2. Promises (`Promise`)

A **Promise** is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value.

A Promise can be in one of three states:

1. **Pending:** The initial state; the operation is still processing.
2. **Fulfilled (Resolved):** The operation completed successfully, and the promise now has a value.
3. **Rejected:** The operation failed, and the promise has an error reason.

Example: Creating a Custom Promise

JavaScript

```
const checkServerStatus = new Promise((resolve, reject) => {
  let isServerUp = true;

  setTimeout(() => {
    if (isServerUp) {
      resolve("Success: Server connection established.");
    } else {
```

NGI/KMEC

```
      reject("Error: Server is down.");
    }
  }, 2000);
});

// Consuming the promise using .then() and .catch()
checkServerStatus
  .then(response => console.log(response))
  .catch(error => console.error(error));
```

3. Modern Asynchronous Flow: `async` and `await`

While `.then()` and `.catch()` chains work, they can become hard to read when dealing with multiple sequential network requests (a problem known as "callback hell").

ES2017 introduced **`async` and `await`**, which allow you to write asynchronous code that looks and behaves like synchronous code.

- **`async`:** Placed before a function declaration to make it return a Promise automatically.
- **`await`:** Pauses the execution of the `async` function until the targeted Promise is settled (fulfilled or rejected).

4. The Fetch API

The **Fetch API** is a built-in browser interface that allows you to make HTTP network requests to servers (similar to `XMLHttpRequest`, but much cleaner and built entirely on Promises).

Real-World Example: Fetching Live Data with `async`/`await`

Let's look at how to fetch live financial exchange data, handle network status validation, and process errors using a `try...catch` block:

JavaScript

```
async function fetchLiveCryptoPrices() {
  console.log("Loading skeleton displayed: Fetching financial data...");

  try {
    // 1. Await the network fetch request
    const response = window.fetch ? await fetch("https://api.coincap.io/v2/assets?limit=3") : null;

    // 2. Validate response headers (HTTP status check)
    if (!response.ok) {
      throw new Error(`HTTP Error! Status: ${response.status}`);
    }

    // 3. Parse response stream into JSON format
    const result = await response.json();
```

NGIT/KMEC

```
    // 4. Transform data using destructuring and map
    const marketSummary = result.data.map(({ id, name, priceUsd }) => ({
      ticker: id.toUpperCase(),
      name,
      price: `$${Number(priceUsd).toFixed(2)}`
    }));

    console.log("Live Crypto Dashboard Updated:", marketSummary);

  } catch (error) {
    console.error("Failed to fetch live prices:", error.message);
  } finally {
    console.log("Loading skeleton removed. UI rendered.");
  }
}

// Execute the async function
fetchLiveCryptoPrices();
```

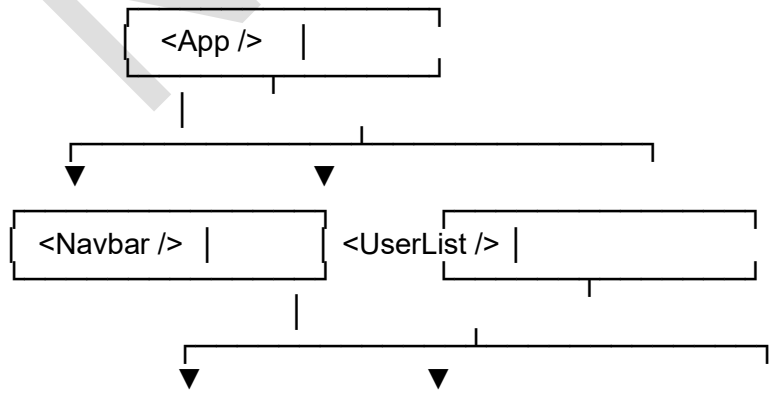
ReactJS Fundamentals & Component Architecture

Having mastered asynchronous JavaScript and the Fetch API, we now step into the core engine of modern frontends: **ReactJS**. React transitions us from manually querying and updating HTML elements to building declarative, component-driven user interfaces.

1. What is React & Why Component-Driven Architecture?

In traditional web development, web pages are built as monolithic HTML documents. Whenever data updates, developers write manual DOM manipulations (e.g., document.getElementById('price').textContent = ...). As applications grow, synchronizing UI with changing data becomes brittle, error-prone, and slow.

React reimagines web applications as trees of **independent, reusable, and self-contained components**.



NGTKMEC



- **Declarative UI:** You declare *what* the interface looks like for a given data state, and React automatically updates the browser display when that state changes.

- **Virtual DOM & Fast Diffing:** Direct DOM updates force the browser to recalculate layouts and repaint the screen. React maintains a lightweight representation of the DOM in memory (the **Virtual DOM**). When data changes, React computes the exact difference (**diffing**) and applies only the necessary changes to the real DOM.

2. JSX (JavaScript XML) Syntax Rules
JSX is a syntax extension for JavaScript that allows you to write HTML-like markup directly inside JavaScript files. Under the hood, JSX compiles to standard React.createElement() calls.

- **Single Root Element:** Every component's return statement must wrap elements inside a single parent tag or a Fragment (<>...</>).

- **Attributes use CamelCase:** HTML attributes become camelCase in JSX (class becomes className, for becomes htmlFor, onclick becomes onClick).

- **Embedding Expressions:** Any valid JavaScript variable, calculation, or function call can be embedded directly into JSX markup using curly braces {}.

JavaScript
```
// Embedding JavaScript expressions inside JSX
const appTitle = "Engineering Portal";
const currentYear = new Date().getFullYear();

function Header() {
  return (
    <header className="main-header">
      <h1>{appTitle}</h1>
      <p>Active Session Year: {currentYear}</p>
    </header>
  );
}
```
3. Component Design: Functional Components
In modern React, components are pure JavaScript functions that accept inputs (called **props**) and return JSX to describe the user interface.

JavaScript
```
// 1. Defining a functional component
function WelcomeBanner() {
  return (
    <div className="banner">
      <h2>Welcome to Day 1 React Lab</h2>
      <p>Building reusable UI components.</p>
    </div>
```

NGIT/KMEC

```
  );
}

// 2. Exporting the component for application use
export default WelcomeBanner;
```

4. Component Communication via Props

Props (short for Properties) are the mechanism by which data flows downward from a parent component to a child component.

- **Unidirectional Data Flow:** Data travels strictly top-down (Parent → Child).
- **Immutability:** Props are **read-only**. A child component must never modify the props passed into it.

[Parent: App Component] ——— (passes props: name, role) ———► [Child: Employee Component]

Step 1: Receiving & Destructuring Props

Instead of accessing props.name or props.department, modern React code uses **object destructuring** directly in the function signature for clean, concise syntax.

JavaScript

```
// Employee.jsx
function Employee({ name, employeeId, department, designation, salary }) {
  return (
    <div className="employee-card">
      <h3>{name}</h3>
      <p><strong>Employee ID:</strong> {employeeId}</p>
      <p><strong>Department:</strong> {department}</p>
      <p><strong>Designation:</strong> {designation}</p>
      <p><strong>Salary:</strong> ₹{salary}</p>
    </div>
  );
}

export default Employee;
```

Step 2: Supplying Props from Parent

JavaScript

```
// App.jsx
import Employee from './Employee';

function App() {
  return (
    <div className="app-container">
      <h1>Company Directory</h1>

      {/* Reusing the same Employee component with different props */}
      <Employee
        name="Aarav Sharma"
        employeeId="EMP101"
```

NGIT/KMEC

```
        department="Engineering"
        designation="Full Stack Developer"
        salary="85,000"
      />
      <Employee
        name="Priya Patel"
        employeeId="EMP102"
        department="Engineering"
        designation="Frontend Lead"
        salary="95,000"
      />
    </div>
  );
}

export default App;
```

5. Dynamic Component Lists with .map() **and** key

When displaying lists of items, React utilizes JavaScript's .map() method to transform an array of data objects into an array of rendered components.

JavaScript

```
function EmployeeList() {
  const staff = [
    { id: "E01", name: "Aarav Sharma", dept: "Engineering", role: "Developer", pay: "80,000" },
    { id: "E02", name: "Priya Patel", dept: "Marketing", role: "Designer", pay: "65,000" },
    { id: "E03", name: "Rohan Verma", dept: "Finance", role: "Analyst", pay: "70,000" }
  ];

  return (
    <div className="staff-grid">
      {staff.map((emp) => (
        <Employee
          key={emp.id} // Essential for React Virtual DOM reconciliation
          name={emp.name}
          employeeId={emp.id}
          department={emp.dept}
          designation={emp.role}
          salary={emp.pay}
        />
      ))}
    </div>
  );
}
```

Why the key **prop matters:** React uses the unique key attribute during Virtual DOM diffing to track which items have been created, modified, or deleted. Always use a stable, unique identifier (like an id) rather than the array index.

NSTKMEC

🧪 Hands-On Lab: React Exercise 1

React Exercise 1: Employee Profile Card

- **Problem Statement:** Create a React application that displays employee information using reusable components. Develop an Employee component that receives Employee Name, Employee ID, Department, Designation, and Salary as props. Render the Employee component multiple times from App.jsx with different employee details. Each employee should be displayed inside a separate card with proper headings and labels. The application should demonstrate component reusability using props.
- **Playwright Test Cases:**
    - Verify page heading.
    - Verify four employee cards.
    - Verify employee names.
    - Verify Employee ID.
    - Verify Department.
    - Verify Designation.
    - Verify Salary.

Solution

Here is the complete step-by-step development process, file structure, code implementation, and execution guidelines.

1. Project Directory Structure

Create a standard Vite React project structure containing our reusable card component:

Plaintext

```
employee-card-app/
├── src/
│   ├── components/
│   │   └── Employee.jsx      # Child Component (Receives & renders props)
│   ├── App.jsx               # Parent Component (Holds data & supplies props)
```

NGIT/KMEC

```
│   ├── App.css            # Clean card styling
│   ├── index.css          # Base reset styles
│   └── main.jsx           # React mounting root
├── index.html
└── package.json
```

2. Step-by-Step Code Implementation

Step 1: Create the Child Component (src/components/Employee.jsx**)**

This component uses ES6 object destructuring directly inside the function parameter list to extract the required props cleanly.

JavaScript

```
// src/components/Employee.jsx
function Employee({ name, employeeId, department, designation, salary }) {
  return (
    <div className="employee-card">
      <h2 className="employee-name">{name}</h2>
      <p className="employee-id"><strong>Employee ID:</strong> {employeeId}</p>
      <p className="employee-dept"><strong>Department:</strong> {department}</p>
      <p className="employee-desig"><strong>Designation:</strong> {designation}</p>
      <p className="employee-salary"><strong>Salary:</strong> ₹{salary}</p>
    </div>
  );
}

export default Employee;
```

Step 2: Create the Parent Component (src/App.jsx**)**

App.jsx provides the page heading, holds the data collection, and reuses the <Employee/> component 4 times with unique props.

JavaScript

```
// src/App.jsx
import Employee from './components/Employee';
import './App.css';

function App() {
  const employees = [
    {
      id: "EMP101",
      name: "Aarav Sharma",
      department: "Engineering",
      designation: "Full Stack Developer",
      salary: "85,000"
    },
    {
      id: "EMP102",
      name: "Priya Patel",
      department: "Engineering",
      designation: "Frontend Lead",
      salary: "95,000"
```

NGIT/KMEC

```
    },
    {
      id: "EMP103",
      name: "Rohan Verma",
      department: "Finance",
      designation: "Financial Analyst",
      salary: "70,000"
    },
    {
      id: "EMP104",
      name: "Ananya Iyer",
      department: "Human Resources",
      designation: "HR Specialist",
      salary: "65,000"
    }
  ];

  return (
    <div className="app-container">
      <h1 className="page-heading">Employee Profile Cards</h1>
      <div className="card-grid">
        {employees.map((emp) => (
          <Employee
            key={emp.id}
            name={emp.name}
            employeeId={emp.id}
            department={emp.department}
            designation={emp.designation}
            salary={emp.salary}
          />
        ))}
      </div>
    </div>
  );
}

export default App;
```

Step 3: Add Styling (src/App.css**)**

CSS

```
/* src/App.css */
.app-container {
  max-width: 1000px;
  margin: 0 auto;
  padding: 2rem;
  font-family: Arial, sans-serif;
  text-align: center;
}

.page-heading {
  color: #1e293b;
  margin-bottom: 2rem;
```

NGIT/KMEC

```
}

.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1.5rem;
}

.employee-card {
  background-color: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
  padding: 1.25rem;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  text-align: left;
}

.employee-name {
  color: #2563eb;
  font-size: 1.25rem;
  margin-top: 0;
  margin-bottom: 0.75rem;
  border-bottom: 1px solid #e2e8f0;
  padding-bottom: 0.5rem;
}

.employee-card p {
  margin: 0.4rem 0;
  color: #475569;
  font-size: 0.95rem;
}
```

3. Execution & Testing Guidelines

1. Initialize the Application

Bash

```
# Create project via Vite
npm create vite@latest employee-card-app -- --template react

# Move into project directory
cd employee-card-app

# Install dependencies
npm install
```

2. Add the Component Files

- Place Employee.jsx inside src/components/.
- Replace src/App.jsx and src/App.css with the code above.

NGIT/KMEC

3. Run the Development Server

Bash
npm run dev
Open your browser at the local server address (usually http://localhost:5173) to view the 4 rendered employee cards.

4. Playwright Test Verification Suite
Create an end-to-end test file (tests/employeeCards.spec.js) to verify all 7 curriculum rubric test points:

JavaScript
```
// tests/employeeCards.spec.js
import { test, expect } from '@playwright/test';

test('Verify Employee Profile Cards implementation', async ({ page }) => {
  await page.goto('http://localhost:5173');

  // 1. Verify page heading
  await expect(page.locator('h1')).toHaveText('Employee Profile Cards');

  // 2. Verify four employee cards
  const cards = page.locator('.employee-card');
  await expect(cards).toHaveCount(4);

  // 3. Verify employee names
  await expect(page.locator('.employee-name').first()).toHaveText('Aarav Sharma');

  // 4. Verify Employee ID
  await expect(page.locator('.employee-id').first()).toContainText('EMP101');

  // 5. Verify Department
  await expect(page.locator('.employee-dept').first()).toContainText('Engineering');

  // 6. Verify Designation
  await expect(page.locator('.employee-desig').first()).toContainText('Full Stack Developer');

  // 7. Verify Salary
  await expect(page.locator('.employee-salary').first()).toContainText('₹85,000');
});
```

NGIT/KMEC